

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторной работы №1
по дисциплине «Теория алгоритмов»

«Реализация двумерного клеточного автомата с окрестностью
фон Неймана»

Вариант 17

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Востров А. В. _____

«_____» _____ 2025г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Математическое описание задачи	4
1.1 Понятие клеточного автомата	4
1.2 Функция перехода клеточного автомата	4
1.3 Граничные условия	6
1.4 Классификация динамического поведения по Вольфраму	7
2 Особенности реализации	9
2.1 Реализация двоичного правила	9
2.2 Класс CellularAutomaton	9
2.3 Графический интерфейс программы	12
2.3.1 Класс CellularAutomatonGUI	12
3 Анализ клеточного автомата по заданному правилу	14
4 Результаты работы программы	18
Заключение	22
Приложение. Реализация графического интерфейса	25

Введение

В отчете представлено описание хода выполнения лабораторной работы по дисциплине «Теория алгоритмов».

В рамках лабораторной работы необходимо реализовать двумерный клеточный автомат с окрестностью фон Неймана в соответствии с полученным номером варианта по формуле:

$$№ = \text{номер варианта} * 11 * \text{год рождения} * \text{день} * \text{месяц}.$$

$$№ = 17 * 11 * 2005 * 6 * 10 = 22496100_{10}$$

Граничные условия автомата – торроидальные. Для пользователя должна быть реализована возможность самостоятельного выбора ширины поля и количества итераций. Также необходимо учесть возможность ввода начальных условий как вручную, так и случайным образом по выбору пользователя.

1 Математическое описание задачи

1.1 Понятие клеточного автомата

Клеточный автомат – это дискретная динамическая модель, представляющая собой сетку произвольной размерности, каждая клетка которой в каждый момент времени может принимать одно из конечного множества состояний, и для которой определено правило перехода клеток из одного состояния в другое.

Формально клеточный автомат можно описать следующим образом:

$$A = (L, S, N, f),$$

где L – решетка клеток, S – конечное множество состояний клетки, N – функция, задающая окрестность для произвольной клетки, f – правило перехода конечного автомата.

Решетка

Решетка (L) – это одномерная, двумерная или n -мерная сетка клеток S . Все клетки внутри решетки одинаковы по своему размеру.

Состояния клетки

Каждая клетка $C_{i,j}$ может находиться в одном из конечного множества состояний $S = s_1, s_2, \dots, s_k$. Состояние каждой клетки определяется заданными правилами перехода. Правила рассчитывают следующее состояние клетки из текущего состояния клетки и состояния соседних клеток.

Окрестность

В работе рассматривается окрестность фон Неймана, в которой в качестве соседних клеток клетки $C_{i,j}$ рассматриваются только те клетки, которые имеют общую границу с данной клеткой (рис. 1).

Инициализация

Начальное состояние автомата описывается функцией $func : L \rightarrow S$. Каждой клетке присваивается её начальное состояние либо случайным образом, либо по вводу пользователя.

1.2 Функция перехода клеточного автомата

Функция перехода клеточного автомата $f : S^{n+1} \rightarrow S$ задает переход от множества всевозможных конфигураций состояний на окрестности размера

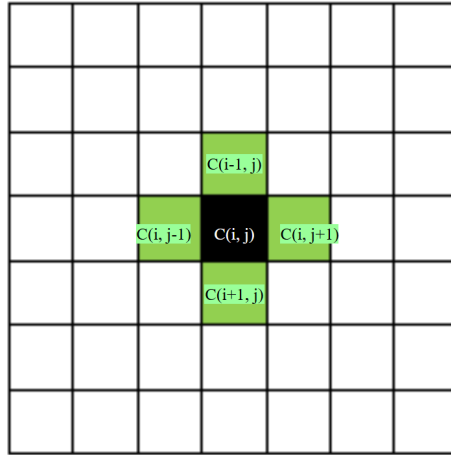


Рис. 1: Окрестность фон Неймана

n к конечному множеству состояний клетки. На каждом шаге t состояние каждой клетки обновляется по следующей формуле:

$$C_{i,j}^{t+1} = f(C_{i,j-1}^t, C_{i-1,j}^t, C_{i,j+1}^t, C_{i+1,j}^t, C_{i,j}^t),$$

где $\forall i, j \in [0, n-1], t \in [0, iter-1], n$ – размер поля, $iter$ – число итераций.

В данном случае f является функцией 5 переменных, что обуславливается выбором окрестности фон Неймана: в ней рассматривается текущая клетка и 4 соседние с ней клетки. Вектор значений функции вычисляется по формуле:

$$f = \text{номер варианта} \times 11 \times \text{год рождения} \times \text{день} \times \text{месяц}.$$

$$f = 17 \times 11 \times 2005 \times 6 \times 10 = 22496100_{10} = 1010101110100001101100100_2.$$

Вектор дополняется незначащими нулями до 32 знаков, чтобы достигнуть размерности $2^5 = 32$. Таблица истинности для функции-правила f приведена в таблице 1.

Таблица 1: Таблица истинности функции f

a	b	c	d	e	$f(a, b, c, d, e)$
0	0	0	0	0	0
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	0
0	0	1	0	0	0
0	0	1	0	1	0
0	0	1	1	0	0
0	0	1	1	1	1

0	1	0	0	0	0
0	1	0	0	1	1
0	1	0	1	0	0
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	0	1	1
1	0	0	1	0	0
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	0	1	0
1	0	1	1	0	1
1	0	1	1	1	1
1	1	0	0	0	0
1	1	0	0	1	1
1	1	0	1	0	1
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	1
1	1	1	1	0	0
1	1	1	1	1	0

1.3 Граничные условия

В лабораторной работе применяются торроидальные граничные условия – тип граничных условий, при которых противоположные грани ячейки объявляют тождественными, в результате чего ячейка замыкается сама на себя и преобразуется в торообразную фигуру, у которой нет границ. В реализованной программе верхняя и нижняя, правая и левая границы сетки являются тождественными друг другу.

Для поля размером $n \times m$ торроидальные граничные условия задаются следующим образом:

- $x_t : x = x \bmod n$;
- $y_t : y = y \bmod m$.

Граничные условия затрагивают ячейки, находящиеся в углах сетки, а также в рядах и столбцах, прилегающих к границам поля. Рассмотрим некоторые из этих случаев:

- верхний ряд клеток: $C_{0,j}^{t+1} = f(C_{0,j-1}^t, C_{n-1,j}^t, C_{0,j+1}^t, C_{1,j}^t, C_{0,j}^t)$;
- нижний ряд клеток: $C_{n-1,j}^{t+1} = f(C_{n-1,j-1}^t, C_{n-2,j}^t, C_{n-1,j+1}^t, C_{0,j}^t, C_{n-1,j}^t)$;
- левый столбец клеток: $C_{i,0}^{t+1} = f(C_{i,m-1}^t, C_{i-1,0}^t, C_{i,1}^t, C_{i+1,0}^t, C_{i,0}^t)$;
- правый столбец клеток: $C_{i,m-1}^{t+1} = f(C_{i,m-2}^t, C_{i-1,m-1}^t, C_{i,0}^t, C_{i+1,m-1}^t, C_{i,m-1}^t)$;

1.4 Классификация динамического поведения по Вольфраму

Динамическое поведение – это характер эволюции во времени, определяемый правилами перехода и начальными условиями. Динамическое поведение описывает, как система развивается от одного шага к следующему и формирует устойчивые, периодические или хаотические структуры.

Сходимость – свойство системы приходить к устойчивому состоянию или циклической конфигурации, которая не изменяется при дальнейших итерациях.

Паттерн – устойчивая конфигурация клеток, возникающая в процессе эволюции. Они могут быть статическими, периодическими или динамическими.

Стивен Вольфрам предложил классификацию, разделив все динамическое поведение клеточных автоматов на четыре класса:

1. Класс I: полная сходимость к однородному состоянию.

Системы характеризуются тенденцией к единообразному воспроизведению одного и того же. Они быстро эволюционируют к однородному, постоянному состоянию и демонстрируют простые паттерны. Таким системам свойственна высокая сходимость. В качестве примера можно привести правила 0 и 255.

2. Класс II: сходимость к простым периодическим или статическим паттернам.

Системы характеризуются простым циклическим поведением. Системы эволюционируют к устойчивым периодическим или статическим паттернам, изолированным друг от друга. В качестве примера таких систем можно привести правила 4, 108, 218 и 250.

3. **Класс III:** хаотическое поведение со сложными непериодическими паттернами.

Системы демонстрируют хаотическое, апериодическое поведение и характеризуются сложными паттернами. Поведение этих систем выглядит случайным, и системы этого класса не сходятся, генерируя постоянно меняющиеся структуры. В качестве примера можно привести правила 30 и 150.

4. **Класс IV:** сложное поведение с возникающими структурами и возможностью универсальных вычислений.

Системы характеризуются незакономерным и непредсказуемым поведением. Такие системы демонстрируют поведение, граничащее между упорядоченным и хаотичным, порождая сложные самовоспроизводящиеся структуры. Такие системы считаются наиболее интересными с точки зрения динамики, так как способны к вычислениям по Тьюрингу. В качестве примера можно привести правило 110.

2 Особенности реализации

2.1 Реализация двоичного правила

Метод `create_rule_function` переводит десятичное правило в двоичное представление, дополняя его незначащими нулями до 32 символов.

Вход: десятичное число – правило перехода клеточного автомата.

Выход: строка, хранящая в себе двоичное правило перехода клеточного автомата.

```
1 def create_rule_function(decimal_rule):
2     binary_rule = bin(decimal_rule)[2:]
3     binary_rule = binary_rule.zfill(32)
4     return binary_rule
```

Листинг 1: Реализация метода `create_rule_function`

2.2 Класс `CellularAutomaton`

Логика поведения двумерного клеточного автомата с окрестностью фон Неймана реализована в классе `CellularAutomaton`. Класс содержит в себе следующие структуры данных:

- `width`, `height` – целочисленные значения, хранящие ширину и высоту поля соответственно;
- `field` – двумерный массив для хранения состояний клеток поля;
- `current_step` – целочисленное значение, хранящее количество произведенных шагов;
- `binary_rule` – строковое значение, хранящее 32-символьное бинарное правило перехода клеточного автомата.

Метод `__init__` является конструктором класса. Он инициализирует объект класса `CellularAutomaton` и все хранимые в нём структуры данных.

Вход: ссылка на инициализируемый объект, ширина поля, высота поля, правило перехода в десятичной системе счисления.

Выход: инициализированный двумерный клеточный автомат.

```
1 def __init__(self, width, height, rule):
2     self.width = width
3     self.height = height
4     self.field = np.zeros((height, width), dtype=int)
5     self.current_step = 0
6     self.rule_binary = create_rule_function(rule)
```

Листинг 2: Реализация метода `__init__`

Метод `random_initialization` случайным образом присваивает клеткам поля состояние «живой» (1) или «мёртвой» (0) клетки.

Вход: ссылка на инициализированный объект.

Выход: поле, случайным образом заполненное состояниями клеток.

```
1 def random_initialization(self):
2     self.field = np.random.choice([0, 1], size=(self.height, self
        .width))
3     self.current_step = 0
```

Листинг 3: Реализация метода `random_initialization`

Метод `clear_field` очищает текущие состояния клеток и сбрасывает поле до исходной конфигурации: все клетки принимают «мёртвое» состояние, а `current_step` присваивается значение 0.

Вход: ссылка на инициализированный объект.

Выход: пустое поле.

```
1 def clear_field(self):
2     self.field = np.zeros((self.height, self.width), dtype=int)
3     self.current_step = 0
```

Листинг 4: Реализация метода `clear_field`

Метод `set_cell_state` предназначен для установки конкретного состояния отдельной клетки клеточного автомата по заданным координатам.

Вход: ссылка на инициализированный объект, координаты `x` и `y` клетки поля, устанавливаемое состояние клетки (0 или 1).

Выход: измененное состояние клетки на поле.

```
1 def set_cell_state(self, x, y, state):
2     if 0 <= x < self.width and 0 <= y < self.height:
3         self.field[y, x] = state
```

Листинг 5: Реализация метода `set_cell_state`

Метод `get_neighbors` возвращает состояния всех соседних клеток для заданной клетки согласно окрестности фон Неймана с торроидальными граничными условиями.

Вход: ссылка на инициализированный объект, координаты `x` и `y` текущей клетки.

Выход: список из 4 целых чисел, представляющих состояния соседних клеток в порядке: [верх, лево, низ, право].

```
1 def get_neighbors(self, x, y):
2     return [
3         self.field[(y - 1) % self.height, x],      # верх
4         self.field[y, (x - 1) % self.width],        # лево
5         self.field[(y + 1) % self.height, x],      # низ
6         self.field[y, (x + 1) % self.width]         # право
7     ]
```

Листинг 6: Реализация метода `get_neighbors`

Метод `get_rule_index` вычисляет индекс правила перехода на основе состояний соседних клеток и текущего состояния целевой клетки. Этот индекс используется для определения следующего состояния клетки согласно двоичному представлению правила.

Вход: ссылка на инициализированный объект, 4 целых чисел, представляющих состояния соседних клеток в порядке: [верх, лево, низ, право], текущее состояние целевой клетки (0 или 1).

Выход: целое число в диапазоне от 0 до 31, представляющее индекс в двоичном правиле перехода.

```
1 def get_rule_index(self, neighbors, current_state):
2     top, left, bottom, right = neighbors
3     binary_string = f"{top}{left}{bottom}{right}{current_state}"
4     return int(binary_string, 2)
```

Листинг 7: Реализация метода `get_rule_index`

Метод `step` выполняет один шаг эволюции клеточного автомата, обновляя состояния всех клеток поля одновременно.

Вход: ссылка на инициализированный объект.

Выход: обновленная конфигурация поля.

```
1 def step(self):
2     new_field = np.zeros((self.height, self.width), dtype=int)
3
4     for y in range(self.height):
5         for x in range(self.width):
6             current_state = self.field[y, x]
7             neighbors = self.get_neighbors(x, y)
8
9             rule_index = self.get_rule_index(neighbors,
10                                              current_state)
11
12             new_state = int(self.rule_binary[rule_index])
13             new_field[y, x] = new_state
14         self.field = new_field
15         self.current_step += 1
```

Листинг 8: Реализация метода `step`

Метод `run` выполняет заданное количество шагов эволюции клеточного автомата, последовательно применяя правило перехода к текущему состоянию поля.

Вход: ссылка на инициализированный объект, количество шагов `steps`.

Выход: обновленная конфигурация поля.

```
1 def run(self, steps):
2     for _ in range(steps):
3         self.step()
```

Листинг 9: Реализация метода `run`

Метод `get_field` возвращает копию текущего состояния поля клеточного автомата.

Вход: ссылка на инициализированный объект.

Выход: двумерный массив целых чисел размером `height × width`, представляющий копию текущего состояния поля.

```
1 def get_field(self):  
2     return self.field.copy()
```

Листинг 10: Реализация метода `get_field`

Метод `get_stats` собирает и возвращает статистическую информацию о текущем состоянии клеточного автомата.

Вход: ссылка на инициализированный объект.

Выход: словарь с ключами `step` (текущий номер шага эволюции), `alive_cells` (количество «живых» клеток), `total_cells` (общее количество клеток), `alive_ratio` (доля живых клеток).

```
1 def get_stats(self):  
2     alive_cells = np.sum(self.field)  
3     total_cells = self.width * self.height  
4     return {  
5         'step': self.current_step,  
6         'alive_cells': alive_cells,  
7         'total_cells': total_cells,  
8         'alive_ratio': alive_cells / total_cells if total_cells >  
9         0 else 0  
    }
```

Листинг 11: Реализация метода `get_stats`

2.3 Графический интерфейс программы

Графический интерфейс программы позволяет пользователю интерактивно управлять клеточным автоматом и наблюдать за его эволюцией. Он реализован с использованием стандартной библиотеки Tkinter.

2.3.1 Класс `CellularAutomatonGUI`

Класс `CellularAutomatonGUI` инкапсулирует всю логику работы графического интерфейса. Код реализации класса приведен в Приложении.

При инициализации создается главное окно размером 1200x800 пикселей.

Компоненты управления

Интерфейс разделен на несколько функциональных областей, они перечислены ниже.

1. Панель параметров:

- поля для ввода ширины и высоты поля (по умолчанию 30x30);
- поле для ввода правила перехода клеточного автомата (по умолчанию 22496100);
- кнопка «Применить» для подтверждения изменений параметров.

2. Панель управления:

- кнопка «Случайное заполнение» для инициализации поля случайными состояниями;
- кнопка «Очистить» для сброса поля на нулевую конфигурацию;
- кнопка «Шаг» для выполнения одного шага эволюции;
- кнопка «Несколько шагов» для выполнения заданного количества шагов;
- поле для ввода количества шагов;
- кнопка «Запуск/Пауза» для запуска и остановки автоматической анимации.

3. Информационная панель:

- отображение текущего номера шага эволюции;
- отображение количества «живых» клеток и их процентное соотношение с общим количеством клеток.

4. Кастомизация: слайдер для изменения размера отображаемых клеток (от 5 до 50 пикселей).

Визуальное представление

Для отображения клеточного поля используется виджет Canvas, на котором рисуется сетка клеток, представляющая собой клеточный автомат. Каждая клетка представляет собой квадрат, цвет которого зависит от ее состояния. Если клетка «мёртвая», то она отображается белым цветом, если клетка «живая», то на её месте отображается специальное изображение.

Обработка ошибок

Реализована проверка вводимых пользователем данных и вывод сообщений об ошибках через стандартные диалоговые окна.

3 Анализ клеточного автомата по заданному правилу

В процессе анализа клеточного автомата проводились тесты переходов состояний по 50 шагов на случайных начальных конфигурациях поля размером 30 на 30 клеток. По результатам анализа были получены сведения, что приблизительно за 10 шагов автомат сходится к статическому состоянию, при котором процент живых клеток составляет около 5-7% от общего количества клеток (см. рис. 2).

На основе проведенного анализа был заключен вывод, что правило 22 496 100 относится к первому классу динамического поведения по классификации Вольфрама, так как клеточный автомат демонстрирует быструю сходимость и в результате своей работы достигает простого, стабильного паттерна.

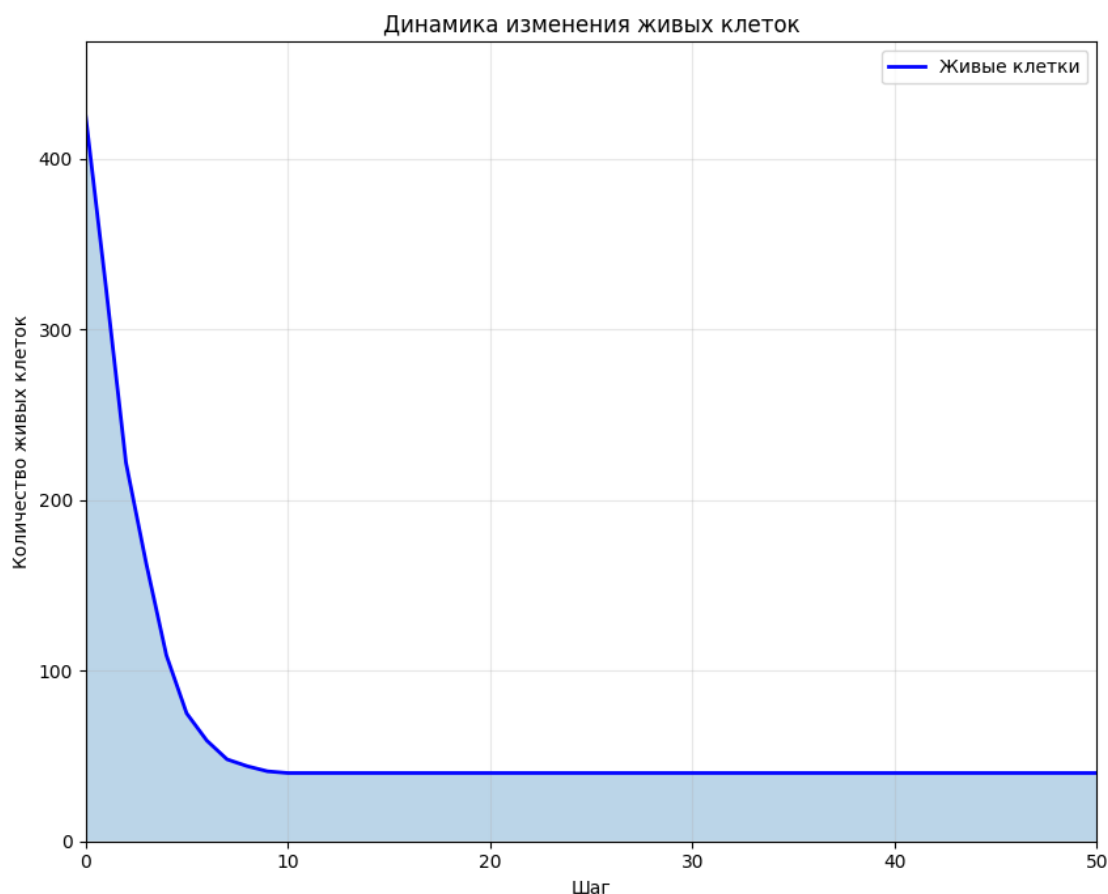


Рис. 2: Статистика изменения количества «живых» клеток

Также было рассмотрено поведение клеточного автомата на начально заданных простых паттернах на поле размером 20x20. Клеточный автомат быстро, менее чем за 10 шагов, приходит к пустому полю. На рисунках 3-11 приведена работа клеточного автомата на таких паттернах, как квадрат, буква «L», крестик.

После запуска работы автомата на поле с квадратом (см. рис. 3), автомат пришел к статическому состоянию за 7 шагов. Клетки, задававшие вертикальные стороны квадрата, стали «мертвыми» на 1-м шаге работы автомата. Далее, слева направо клетки поочередно переходили в «мертвое» состояние, пока поле не опустело (см. рис. 4, 5).

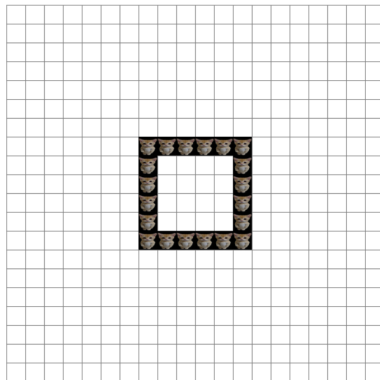


Рис. 3: Начальное состояние поля «квадрат»

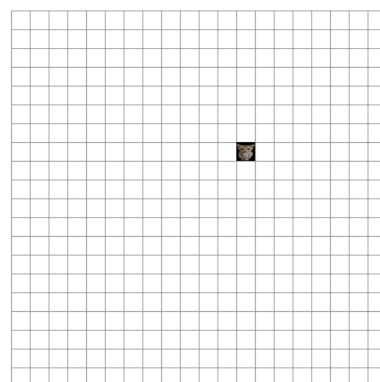
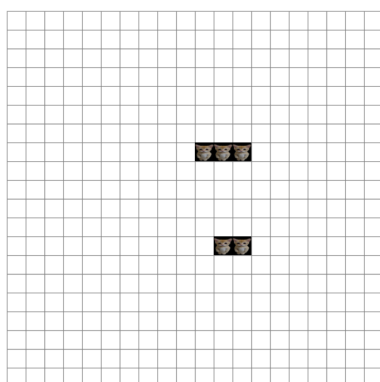
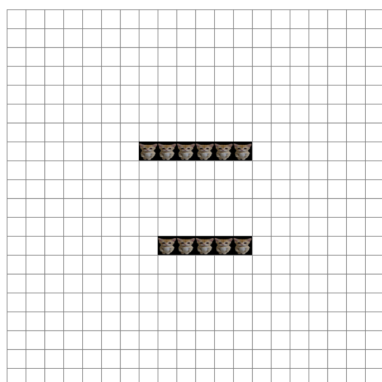


Рис. 4: Шаги 1, 4 и 6 для паттерна «квадрат»

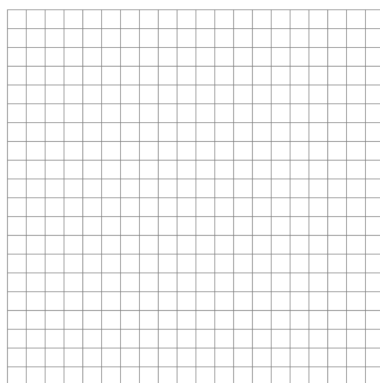


Рис. 5: Шаг 7 для паттерна «квадрат»

После запуска на поле с нарисованной на нем буквой «L» (см. рис. 6) автомат пришел к статическому состоянию за 6 шагов. Аналогично предыдущему примеру, вертикальный ряд «живых» клеток перешел в «мертвое» состояние на 1-м шаге работы, а горизонтальный ряд клеток постепенно переходил в «мертвое» состояние, пока поле не опустело (см. рис. 7, 8).

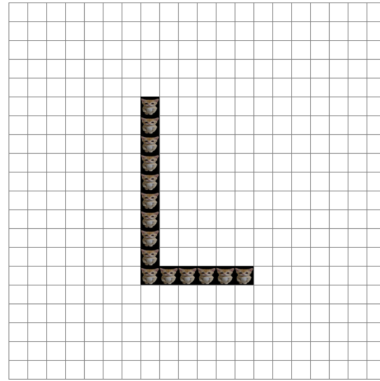


Рис. 6: Начальное состояние поля «L»

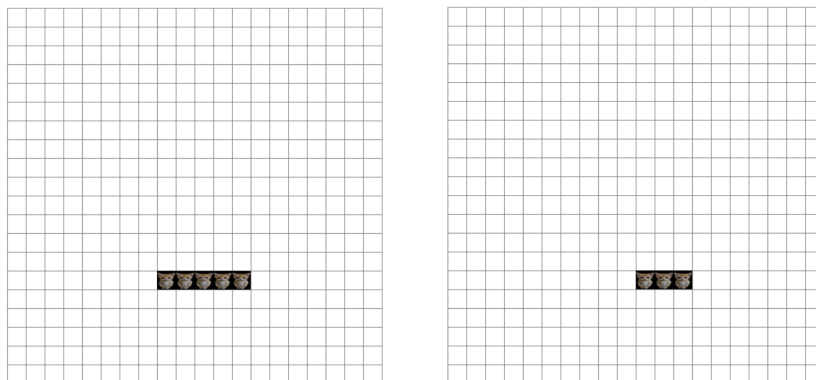


Рис. 7: Шаги 1, 3 для паттерна «L»

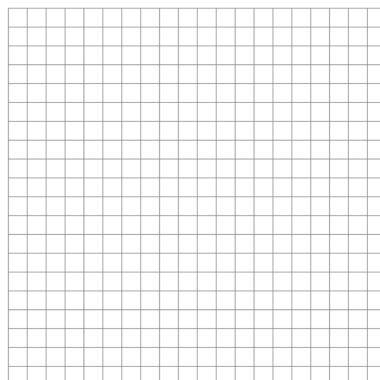


Рис. 8: Шаг 6 для паттерна «L»

Паттерн «крестик» (рис. 9) также продемонстрировал очень быстрое сходжение к пустому полю. Всего за 4 шага клеточный автомат пришел к статическому состоянию (см. рис. 10, 11).

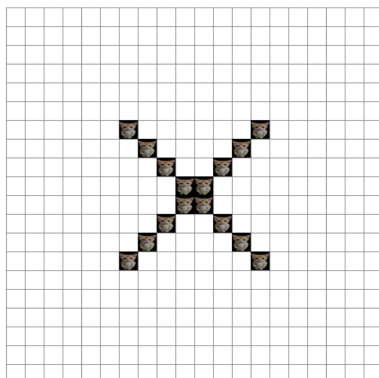


Рис. 9: Начальное состояние поля «крестик»

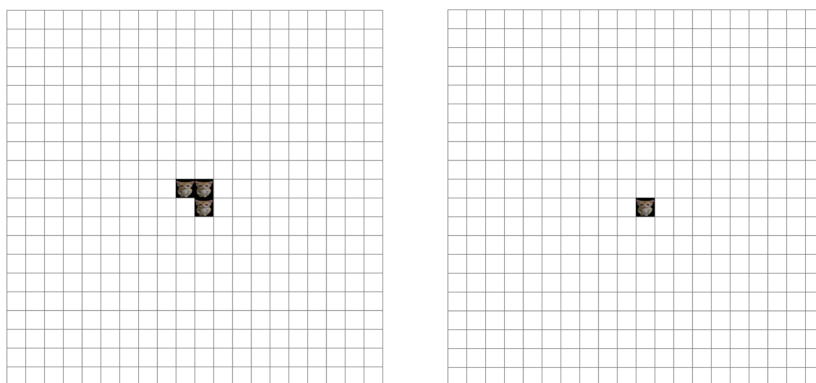


Рис. 10: Шаги 1, 3 для паттерна «крестик»

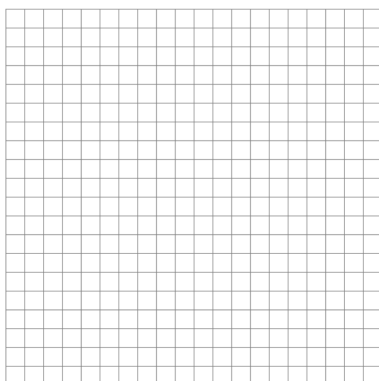


Рис. 11: Шаг 4 для паттерна «крестик»

4 Результаты работы программы

После запуска программы открывается окно с панелью управления и с пустым полем клеточного автомата (рис. 12). Параметры размеров поля и правило перехода уже заданы по умолчанию.

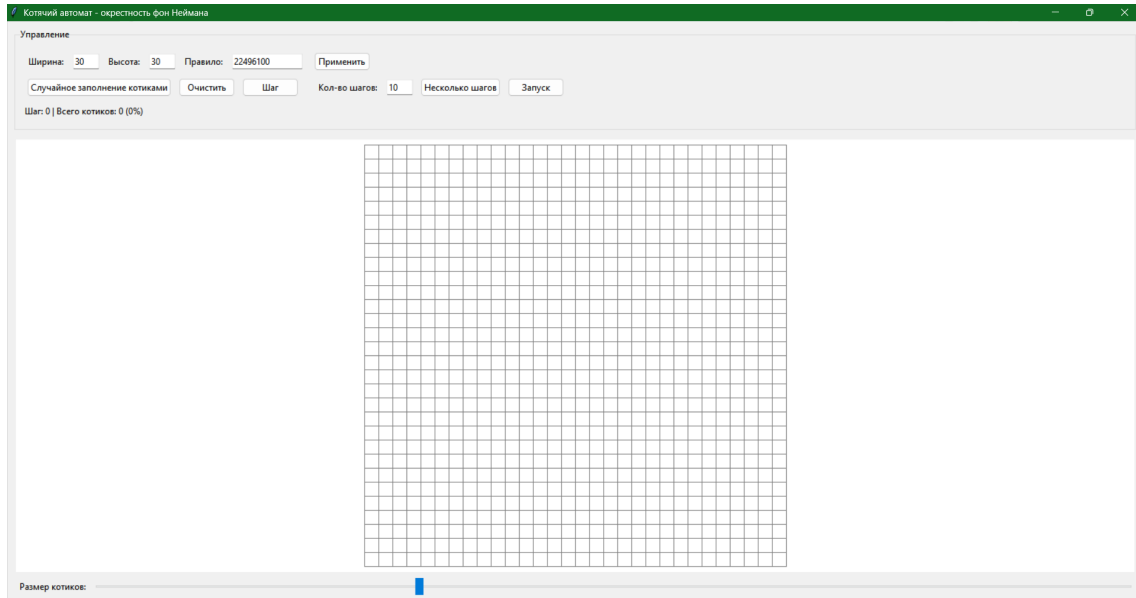


Рис. 12: Окно программы сразу после запуска

Нажимая на клетку поля, пользователь может менять её состояние. Таким образом можно заполнить поле вручную (рис. 13).

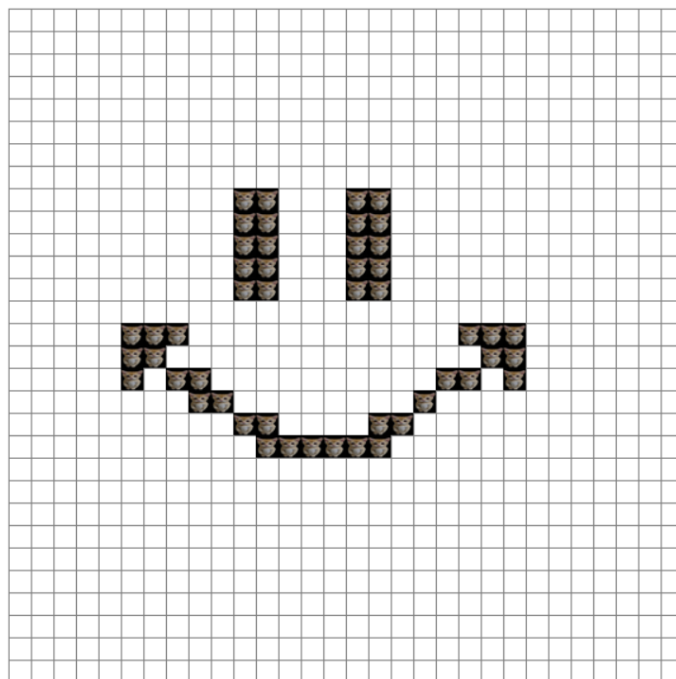


Рис. 13: Заполнение поля вручную

Пользователь может задать другие размеры поля и заполнить поле автоматически. Автоматически заполненное поле размером 15x15 приведено на рис. 14.

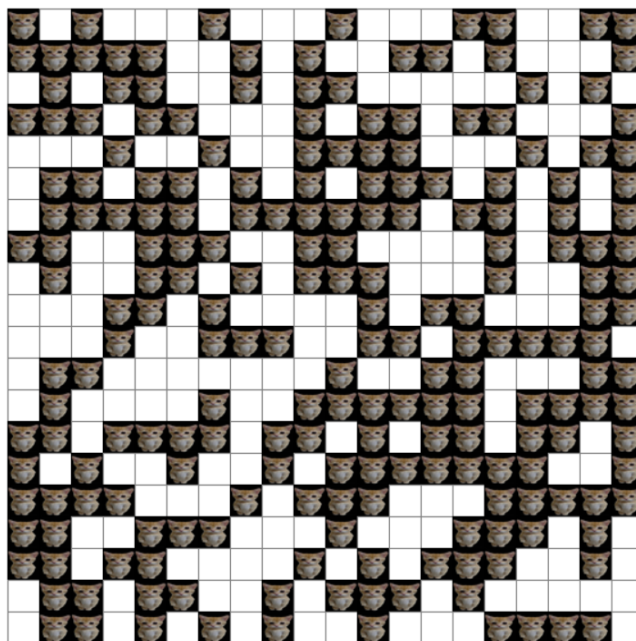


Рис. 14: Заполнение поля автоматически

Пошаговая эволюция клеточного автомата представлена на рисунках 15-20. Наблюдается равномерное уменьшение количества «живых» клеток, пока их отношение к общему количеству клеток не достигает приблизительно 10%. После этого количество «живых» клеток начинает снижаться с меньшей интенсивностью, пока система не достигнет статического состояния.

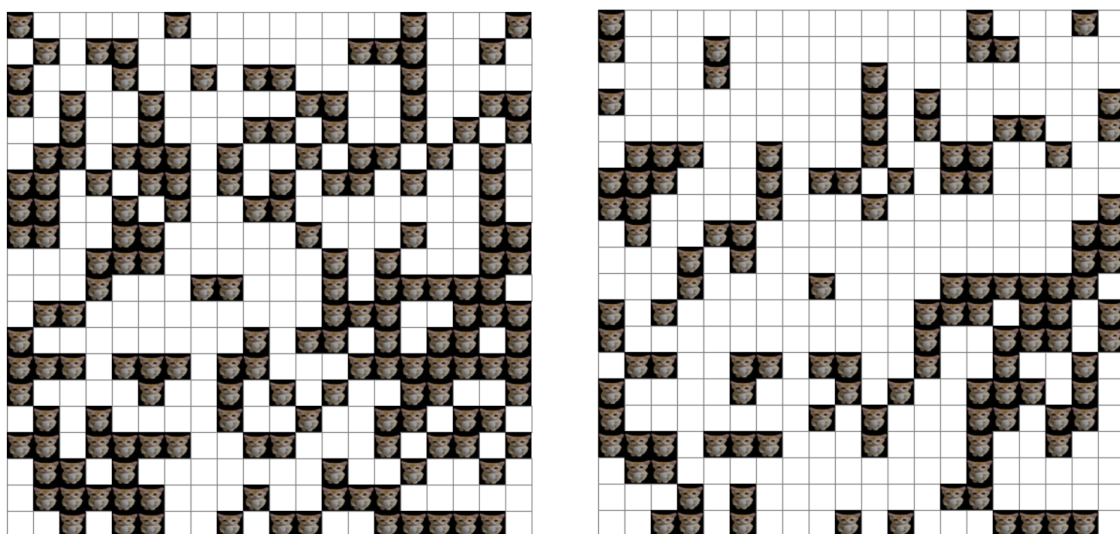


Рис. 15: Шаги 1-2

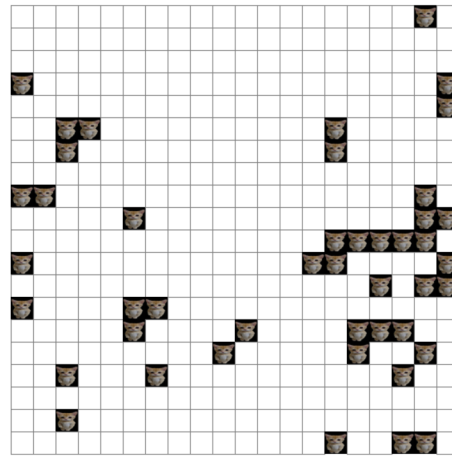
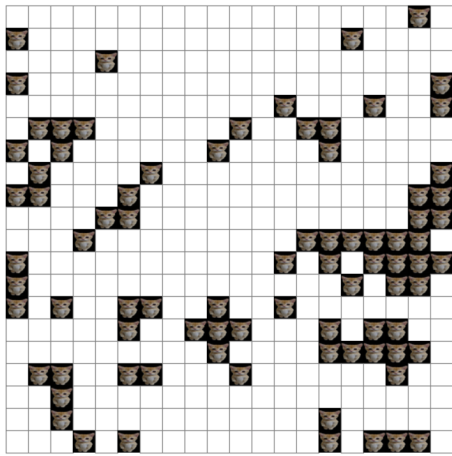


Рис. 16: Шаги 3-4

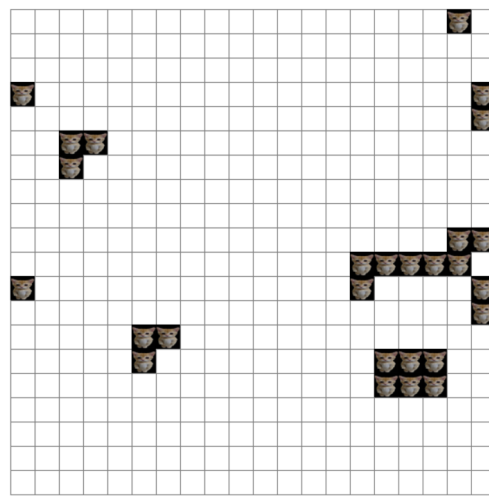
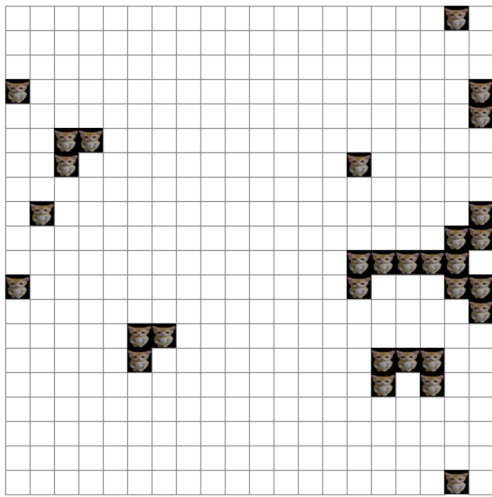


Рис. 17: Шаги 5-6

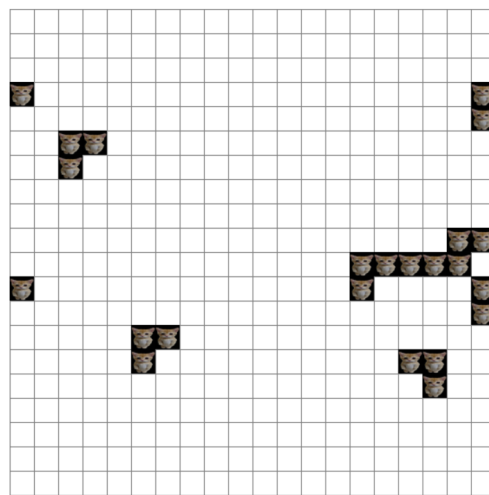
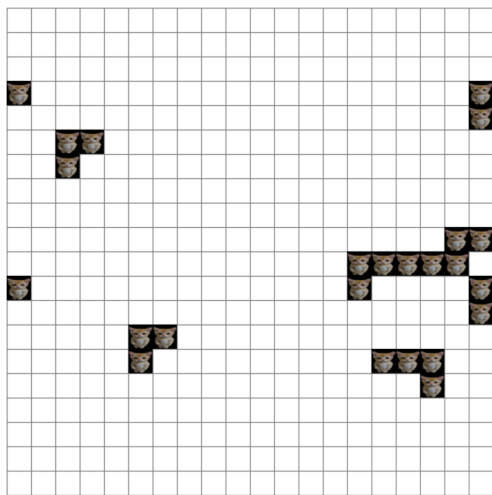


Рис. 18: Шаги 7-8

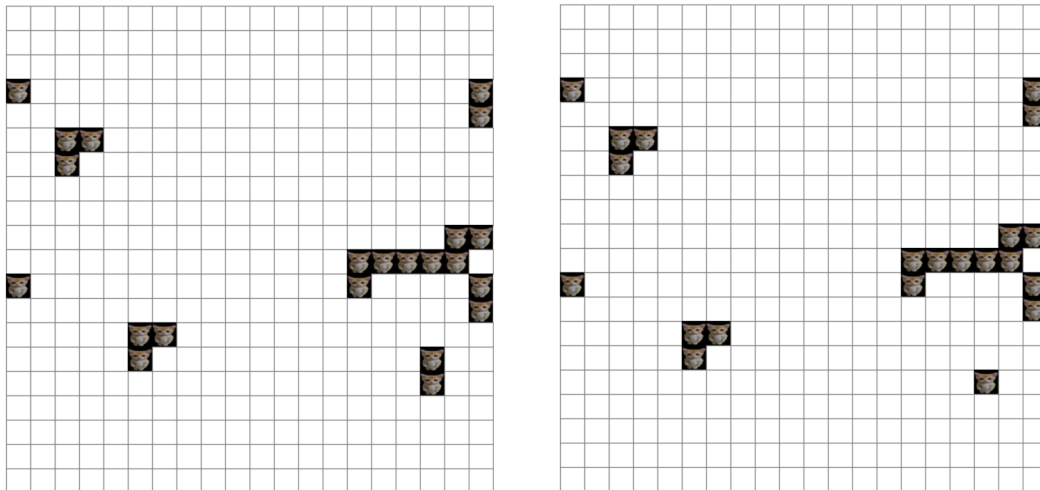


Рис. 19: Шаги 9-10

Пройдя 11 шагов, автомат приходит к статическому состоянию и при дальнейших шагах не изменяет своего состояния (см. рис. 20),

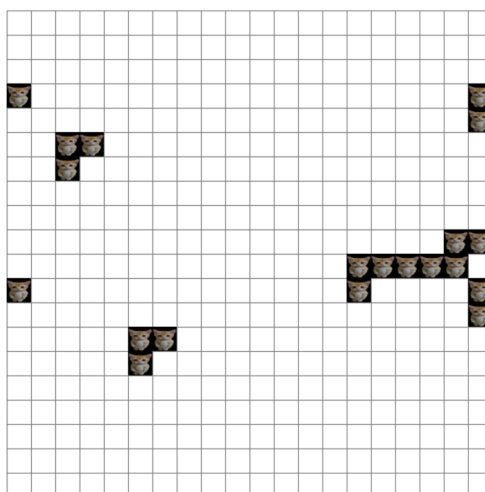


Рис. 20: Шаг 11

Если пользователь введет некорректное значение в поля для ввода, то будет выведено окно с сообщением о соответствующей ошибке (см. рис. 21).

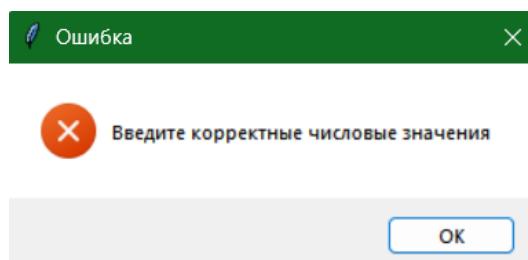


Рис. 21: Сообщение об ошибке

Заключение

В результате выполнения лабораторной работы была реализована программа, описывающая поведение двумерного клеточного автомата с окрестностью фон Неймана и торроидальными граничными условиями для заданного правила перехода 22496100_{10} . Описанный класс `CellularAutomaton` реализует логику поведения двумерного клеточного автомата. В классе `CellularAutomatonGUI` реализованы пользовательский интерфейс программы и визуализация клеточного автомата. Программа предусматривает как ручной ввод ширины и высоты поля, количества итераций и начальных условий, так и автоматическое заполнение. Были проведены эксперименты на разных начальных конфигурациях для анализа поведения клеточного автомата с данным правилом перехода. В результате было заключено, что система относится к первому классу поведения по классификации Вольфрама, что подтверждает график сходимости (рис. 2).

Достоинства программы

1. Программа реализована в объектно-ориентированном стиле, что позволяет легко её масштабировать.
2. Реализован графический интерфейс с визуализацией клеточного автомата, что позволяет наглядно наблюдать поведение автомата и удобно управлять им.
3. Программа предусматривает ввод своего правила перехода клеточного автомата, а не только предусмотренное вариантом работы.
4. Предусмотрена настройка масштаба поля клеточного автомата с помощью слайдера.

Недостатки программы

1. Программа демонстрирует слабую производительность на больших размерах поля от 100×100 и выше.
2. Не предусмотрен выбор окрестности автомата, в программе реализована только окрестность фон Неймана.
3. Не реализовано построение графиков динамики изменения состояния автомата.
4. Класс `CellularAutomatonGUI` сильно перегружен: он совмещает в себе и UI, и обработку событий, и анимацию.

Масштабирование программы

1. Реализация выбора окрестности автомата (например, окрестность Мура).
2. Реализация многопоточного вычисления состояния для больших полей.
3. Реализация выбора цвета или изображения для отображения «живых» клеток.

Работа была выполнена в среде разработки PyCharm 2025.2.1.1 (Community Edition) на языке Python.

Список литературы

- [1] Секция: «Телематика» [Электронный ресурс]. – URL: <https://tema.spbstu.ru/algorithm/> (дата обращения: 15.10.2025).
- [2] Клеточные автоматы. Игра «Жизнь». Часть 1 [Электронный ресурс] - Habr. - URL: <https://habr.com/ru/articles/737672/> (дата обращения: 15.10.2025)

Приложение. Реализация графического интерфейса

```
1 import tkinter as tk
2 from tkinter import ttk, messagebox
3 import numpy as np
4 from PIL import Image, ImageTk
5 import pygame
6 pygame.mixer.init()
7
8 class CellularAutomatonGUI:
9     def __init__(self, root):
10         self.root = root
11         self.root.title("Котячий автомат - окрестность фон Неймана")
12         self.root.geometry("1200x800")
13
14         self.width = 30
15         self.height = 30
16         self.cell_size = 20
17         self.rule = 22496100
18
19         try:
20             self.cat_image_original = Image.open("C:/Users/halva/
21             uni/CellularAutomaton/cat.jpg")
22         except FileNotFoundError:
23             messagebox.showwarning("Предупреждение", "Файл 'cat.
24             jpg' не найден. Будут использоваться чёрные клетки
25             .")
26             self.cat_image_original = None
27         self.cat_photo = None
28
29         try:
30             self.meow_sound = pygame.mixer.Sound("C:/Users/halva/
31             uni/CellularAutomaton/meow2.wav")
32         except pygame.error:
33             messagebox.showwarning("Предупреждение", "Файл 'meow.
34             wav' не найден или повреждён. Звук отключён.")
35             self.meow_sound = None
36         except Exception:
37             self.meow_sound = None
38
39         self.offset_x = 0
40         self.offset_y = 0
41         self.is_running = False
42         self.after_id = None
43
44         self.automaton = CellularAutomaton(self.width, self.
45         height, self.rule)
46         self.setup_ui()
47         self.draw_grid()
```

```

42
43 def play_meow(self):
44     if self.meow_sound is not None:
45         self.meow_sound.play()
46
47 def setup_ui(self):
48     main_frame = ttk.Frame(self.root, padding="10")
49     main_frame.pack(fill=tk.BOTH, expand=True)
50
51     control_frame = ttk.LabelFrame(main_frame, text="Управлен
52     ие", padding="10")
53     control_frame.pack(fill=tk.X, pady=(0, 10))
54
55     params_frame = ttk.Frame(control_frame)
56     params_frame.pack(fill=tk.X, pady=5)
57
58     ttk.Label(params_frame, text="Ширина:").grid(row=0,
59     column=0, padx=5)
60     self.width_entry = ttk.Entry(params_frame, width=5)
61     self.width_entry.insert(0, str(self.width))
62     self.width_entry.grid(row=0, column=1, padx=5)
63
64     ttk.Label(params_frame, text="Высота:").grid(row=0,
65     column=2, padx=5)
66     self.height_entry = ttk.Entry(params_frame, width=5)
67     self.height_entry.insert(0, str(self.height))
68     self.height_entry.grid(row=0, column=3, padx=5)
69
70     ttk.Label(params_frame, text="Правило:").grid(row=0,
71     column=4, padx=5)
72     self.rule_entry = ttk.Entry(params_frame, width=15)
73     self.rule_entry.insert(0, str(self.rule))
74     self.rule_entry.grid(row=0, column=5, padx=5)
75
76     ttk.Button(params_frame, text="Применить", command=self.
77     apply_parameters).grid(row=0, column=6, padx=10)
78
79     buttons_frame = ttk.Frame(control_frame)
80     buttons_frame.pack(fill=tk.X, pady=5)
81
82     ttk.Button(buttons_frame, text="Случайное заполнение коти
83     камми", command=self.random_fill).pack(side=tk.LEFT,
84     padx=5)
85     ttk.Button(buttons_frame, text="Очистить", command=self.
86     clear_grid).pack(side=tk.LEFT, padx=5)
87     ttk.Button(buttons_frame, text="Шаг", command=self.step).
88     pack(side=tk.LEFT, padx=5)
89
90     ttk.Label(buttons_frame, text="Кол-во шагов:").pack(side=
91     tk.LEFT, padx=(20, 5))
92     self.steps_entry = ttk.Entry(buttons_frame, width=5)
93     self.steps_entry.insert(0, "10")

```

```

84     self.steps_entry.pack(side=tk.LEFT, padx=5)
85
86     ttk.Button(buttons_frame, text="Несколько шагов", command
87               =self.multiple_steps).pack(side=tk.LEFT, padx=5)
88
89     self.run_pause_button = ttk.Button(buttons_frame, text="3
90               апуска", command=self.toggle_animation)
91     self.run_pause_button.pack(side=tk.LEFT, padx=5)
92
93     info_frame = ttk.Frame(control_frame)
94     info_frame.pack(fill=tk.X, pady=5)
95     self.info_label = ttk.Label(info_frame, text="Шаг: 0 | Вс
96               его котиков: 0 (0%)")
97     self.info_label.pack(side=tk.LEFT)
98
99     self.canvas_frame = ttk.Frame(main_frame)
100    self.canvas_frame.pack(fill=tk.BOTH, expand=True)
101
102    self.canvas = tk.Canvas(self.canvas_frame, bg='white')
103    self.canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
104    self.canvas.bind("<Button-1>", self.on_cell_click)
105    self.canvas.bind("<Configure>", self.on_canvas_configure)
106
107    scale_frame = ttk.Frame(main_frame)
108    scale_frame.pack(fill=tk.X, pady=5)
109    ttk.Label(scale_frame, text="Размер котиков: ").pack(side=
110              tk.LEFT, padx=5)
111    self.scale_var = tk.IntVar(value=self.cell_size)
112    scale_slider = ttk.Scale(scale_frame, from_=5, to=50,
113                          variable=self.scale_var,
114                          command=self.on_scale_change,
115                          orient=tk.HORIZONTAL)
116    scale_slider.pack(side=tk.LEFT, fill=tk.X, expand=True,
117                    padx=5)
118
119    def on_scale_change(self, value):
120        self.cell_size = int(float(value))
121        if self.cat_image_original is not None:
122            resized = self.cat_image_original.resize((self.
123              cell_size, self.cell_size), Image.LANCZOS)
124            self.cat_photo = ImageTk.PhotoImage(resized)
125            self.draw_grid()
126
127    def toggle_animation(self):
128        if not self.is_running:
129            self.start_animation()
130        else:
131            self.stop_animation()
132
133    def start_animation(self):
134        self.is_running = True
135        self.run_pause_button.config(text="Пауза")

```

```

128         self.animate_step()
129
130     def stop_animation(self):
131         self.is_running = False
132         self.run_pause_button.config(text="Занульс")
133         if self.after_id:
134             self.canvas.after_cancel(self.after_id)
135             self.after_id = None
136
137     def animate_step(self):
138         if self.is_running:
139             self.step()
140             self.after_id = self.canvas.after(100, self.
                animate_step)
141
142     def calculate_offsets(self):
143         canvas_width = self.canvas.winfo_width()
144         canvas_height = self.canvas.winfo_height()
145         field_width = self.width * self.cell_size
146         field_height = self.height * self.cell_size
147         self.offset_x = max(0, (canvas_width - field_width) // 2)
148         self.offset_y = max(0, (canvas_height - field_height) //
            2)
149
150     def on_canvas_configure(self, event=None):
151         self.draw_grid()
152
153     def apply_parameters(self):
154         if self.is_running:
155             self.stop_animation()
156
157         try:
158             new_width = int(self.width_entry.get())
159             new_height = int(self.height_entry.get())
160             new_rule = int(self.rule_entry.get())
161
162             if new_width <= 0 or new_height <= 0:
163                 messagebox.showerror("Ошибка", "Размеры должны бы
                    ть положительными числами")
164                 return
165
166             self.width = new_width
167             self.height = new_height
168             self.rule = new_rule
169
170             self.automaton = CellularAutomaton(self.width, self.
                height, self.rule)
171
172             if self.cat_image_original is not None:
173                 resized = self.cat_image_original.resize((self.
                    cell_size, self.cell_size), Image.LANCZOS)
174                 self.cat_photo = ImageTk.PhotoImage(resized)

```

```

175         self.draw_grid()
176         self.update_info()
177
178     except ValueError:
179         messagebox.showerror("Ошибка", "Введите корректные чи
180             словые значения")
181
182     def draw_grid(self):
183         self.canvas.delete("all")
184         self.calculate_offsets()
185
186         for y in range(self.height):
187             for x in range(self.width):
188                 x1 = self.offset_x + x * self.cell_size
189                 y1 = self.offset_y + y * self.cell_size
190                 x2 = x1 + self.cell_size
191                 y2 = y1 + self.cell_size
192
193                 cell_value = self.automaton.field[y][x]
194
195                 if cell_value == 1:
196                     if self.cat_photo is not None:
197                         self.canvas.create_image(x1, y1, anchor=
198                             tk.NW, image=self.cat_photo)
199                     else:
200                         self.canvas.create_rectangle(x1, y1, x2,
201                             y2, fill='black', outline='gray',
202                             width=1)
203                 else:
204                     self.canvas.create_rectangle(x1, y1, x2, y2,
205                         fill='white', outline='gray', width=1)
206
207     def on_cell_click(self, event):
208         if self.is_running:
209             self.stop_animation()
210
211             canvas_x = event.x - self.offset_x
212             canvas_y = event.y - self.offset_y
213             x = int(canvas_x / self.cell_size)
214             y = int(canvas_y / self.cell_size)
215
216             if 0 <= x < self.width and 0 <= y < self.height:
217                 current_value = self.automaton.field[y][x]
218                 self.automaton.field[y][x] = 1 - current_value
219                 self.draw_grid()
220                 self.update_info()
221
222     def random_fill(self):
223         if self.is_running:
224             self.stop_animation()
225         self.automaton.random_initialization()

```

```

222         self.draw_grid()
223         self.update_info()
224
225     def clear_grid(self):
226         if self.is_running:
227             self.stop_animation()
228             self.automaton.clear_field()
229             self.draw_grid()
230             self.update_info()
231
232     def step(self):
233         old_field = self.automaton.get_field()
234         self.automaton.step()
235         new_field = self.automaton.get_field()
236
237         died = False
238         for y in range(self.height):
239             for x in range(self.width):
240                 if old_field[y, x] == 1 and new_field[y, x] == 0:
241                     died = True
242                     break
243             if died:
244                 break
245
246         if died:
247             self.play_meow()
248
249         self.draw_grid()
250         self.update_info()
251
252     def multiple_steps(self):
253         if self.is_running:
254             self.stop_animation()
255
256         try:
257             steps = int(self.steps_entry.get())
258             if steps <= 0:
259                 messagebox.showerror("Ошибка", "Количество шагов
260                                     должно быть положительным")
261                 return
262
263             self.automaton.run(steps)
264             self.draw_grid()
265             self.update_info()
266
267         except ValueError:
268             messagebox.showerror("Ошибка", "Введите корректное чи
269                                     сло шагов")
270
271     def update_info(self):
272         stats = self.automaton.get_stats()
273         percent = stats['alive_ratio'] * 100

```

```
272     self.info_label.config(  
273         text=f"Шаг: {stats['step']} | "  
274         f"Живых клеток: {stats['alive_cells']} ({percent  
           :.1f}%) "  
275     )
```

Листинг 12: Реализация графического интерфейса